LangChain

LangChain 활용법

목차

1

LangChain 개요

LangChain 프레임워크의 기본 개념과 구성 요소를 다룬다.

2

프롬프트 엔지니어링

효과적인 프롬프트 설계 및 체인 구성 방법을 탐구한다.

3

메모리 시스템

대화형 AI에서 맥락 유지를 위한 메모리 구현을 설명한다.

4

문서 처리 시스템

다양한 형식의 문서 로딩 및 처리 방법을 제시한다.

5

벡터 데이터베이스

문서 임베딩 및 검색 기반 시스템 구축을 학습한다.

6

Tool과 Agent

지능형 의사결정 에이전트 개발과 외부 도구 연동 방안을 모색한다.

7

복합 시스템 설계

실제 비즈니스 문제 해결을 위한 통합 솔루션 구축을 다룬다.

LangChain 이해

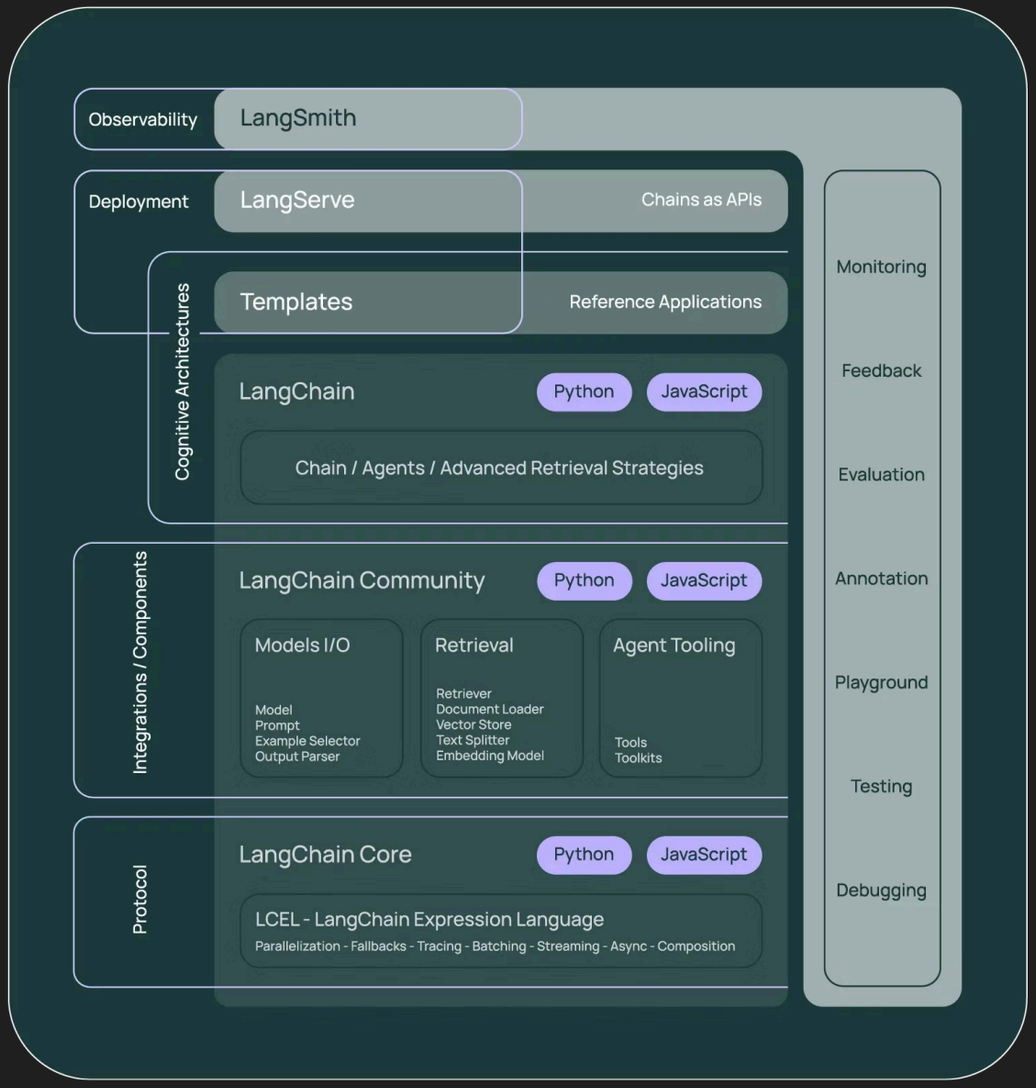
LangChain이란?

대규모 언어 모델(LLM) 기반 애플리케이션 개발을 간소화하는 오픈소스 프레임워크

- 주로 파이썬 및 자바스크립트로 구현
- 복잡한 LLM 시스템 구축을 위한 다양한 컴포넌트와 패턴 제공

주요 특징:

- 모듈화된 컴포넌트로 확장성 용이
- 외부 데이터 소스와의 원활한 통합
- 다양한 LLM 제공업체 지원 (OpenAI, Anthropic, HuggingFace 등)
- 상태 관리 및 메모리 시스템 내장



LangChain의 핵심 구성 요소

LLMs & Chat Models

- 대규모 언어 모델과의 상호작용을 위한 표준화된 인터페이스를 제공
- 텍스트 생성 모델(GPT-4, Claude, Llama 등)과 채팅 기반 모델 인터페이스를 포함

Chains

- 여러 컴포넌트를 연결하여 특정 작업을 순서대로 실행
- 단일 작업 수행 체인이나 여러 체인의 연결(Sequential Chain)이 가능

Agents & Tools

- 자율적인 의사결정 주체로, 특정 도구(Tool) 사용을 결정하고 ReAct 패턴을 통해 추론 및 행동 수행

Memory

- 대화의 맥락과 상태 유지
- 대화 기록을 저장하고 관리하며, 장기 및 단기 메모리 시스템 포함

실제 산업에서의 LangChain 활용 사례

지능형 검색 엔진

의미 기반 검색 및 자연어 질의응답 시스템 구현

맞춤형 고객 지원 챗봇

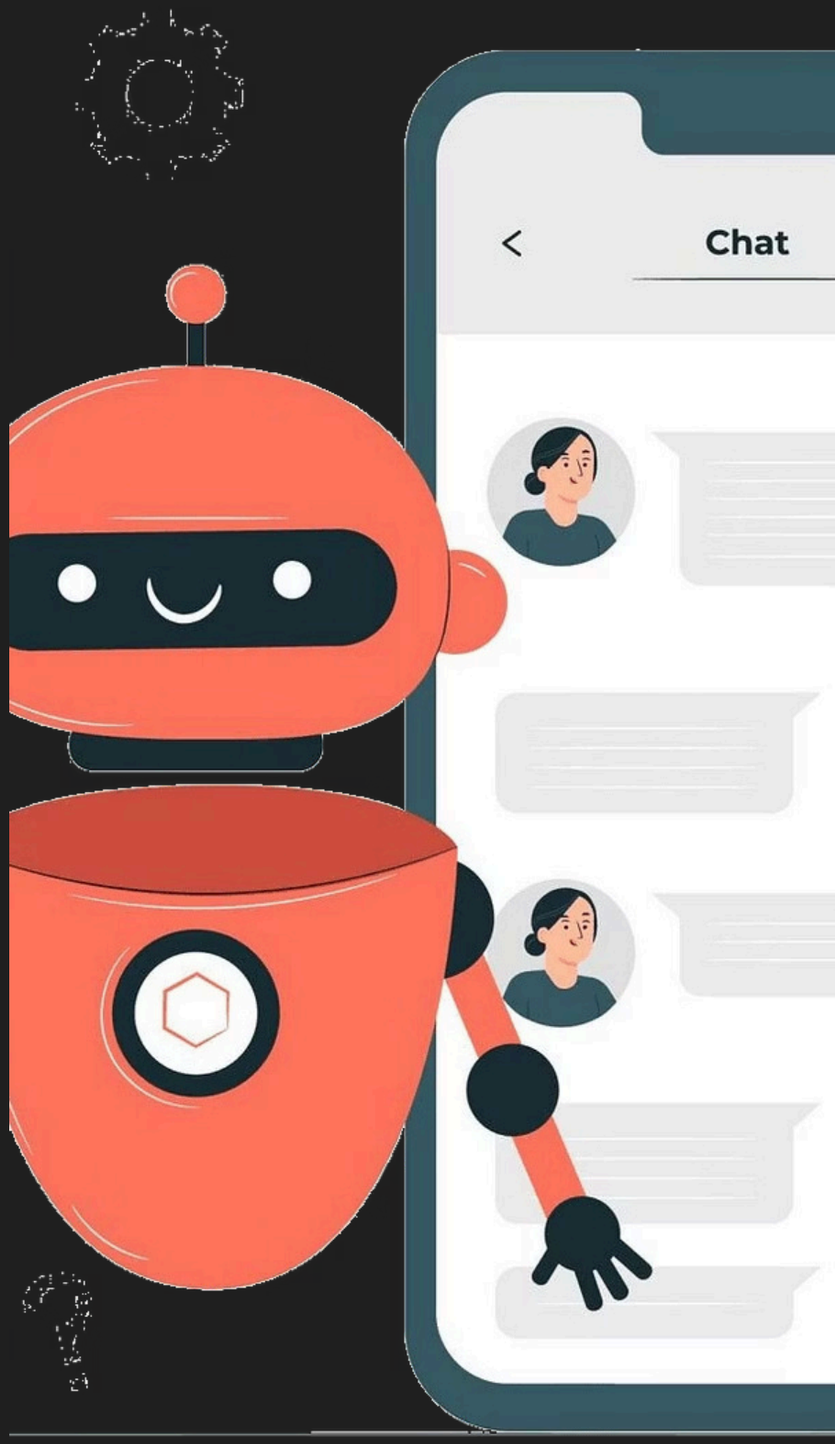
기업 지식 기반 고객 문의 응대 챗봇 구축

데이터 분석 자동화

자연어 기반 데이터 분석 및 인사이트 추출

콘텐츠 생성 및 요약

문서 요약 및 자동 콘텐츠 생성 도구 개발



LangChain 개발 환경 설정

Python 환경 구성

```
# 가상환경 생성 (conda 사용 시)
conda create -n langchain python=3.10
conda activate langchain

# 또는 venv 사용 시
python -m venv langchain_env
source langchain_env/bin/activate  # Windows: langchain_env\Scripts\activate

# 필수 라이브러리 설치
pip install langchain openai chromadb faiss-cpu
```

필수 라이브러리:

- **langchain**: 코어 프레임워크
- **openai**: OpenAI API 연동
- **chromadb/faiss-cpu**: 벡터 데이터베이스

첫 번째 LangChain 애플리케이션

간단한 GPT 호출 (Hello, LangChain!)

```
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

# OpenAI API 키를 설정한다.
import os
os.environ["OPENAI_API_KEY"] = "sk-your-api-key"

# LLM을 초기화한다.
llm = OpenAI(temperature=0.7)

# 프롬프트 템플릿을 정의한다.
template = "안녕하세요! 저는 {name}입니다. {topic}에 대해 알려주세요."
prompt = PromptTemplate(template=template, input_variables=["name", "topic"])

# 체인을 생성하고 실행한다.
chain = LLMChain(llm=llm, prompt=prompt)
response = chain.run(name="한국 개발자", topic="LangChain")

print(response)
```

프롬프트 엔지니어링

PromptTemplate의 이해

PromptTemplate은 LLM에 전달할 입력을 구조화된 형식으로 만든다. 입력 형식의 일관성 유지

- 재사용 가능한 프롬프트 패턴 생성
- 사용자 입력과 시스템 지시 분리

LLM 응답의 품질과 일관성을 크게 향상

```
# 기본 프롬프트 템플릿
from langchain.prompts import
PromptTemplate

template = """
당신은 전문 {profession}입니다.
{topic}에 관해 {tone} 스타일로
설명해주세요.
"""
```

```
prompt = PromptTemplate(
 template=template,
 input_variables=["profession",
 "topic",
 "tone"]
)

formatted = prompt.format(
 profession="AI 연구원",
 topic="트랜스포머 모델",
 tone="친근한"
)
```

고급 프롬프트 패턴

역할 기반 프롬프팅

특정 역할을 부여해 전문성 있는 응답을 유도한다.

```
template = "당신은 {role}입니다. {query}"
```

Few-Shot 학습

예시를 통해 학습시켜 원하는 형태의 응답을 유도한다.

```
template = """
입력: 날씨가 좋아요
출력: 오늘 날씨가 정말 좋네요!

입력: {query}
출력:"""
```

Chain-of-Thought

단계적 사고 과정을 유도해 복잡한 문제를 해결한다.

```
template = """
{query}에 대해 단계별로 생각해보세요.
1. 우선...
2. 다음으로...
3. 마지막으로...
결론:"""
```

체인(Chain)의 개념과 종류

체인은 LangChain의 핵심 컴포넌트다. 여러 단계의 처리 과정을 연결해 복잡한 작업을 수행한다.

LLMChain

- **개념**
  - PromptTemplate과 LLM을 연결하여 결과 텍스트를 생성하는 가장 기본적인 체인
  - 입력 변수 → 템플릿 변환 → LLM 호출 → 출력 반환
- **동작 방식**
  - 사용자 입력 수집
  - PromptTemplate에 입력 변수 적용 (e.g. `"주제: {topic}"`)
  - LLM에 전달 → 결과 생성
  - 결과를 사용자에게 반환
- **활용 예시**
  - 기사 요약하기
  - 면접 질문 자동 생성기
  - 글쓰기 보조 시스템

SequentialChain

- **개념**
  - 여러 개의 체인을 순차적으로 실행하며, **앞선 체인의 출력이 다음 체인의 입력으로 전달**되는 구조
  - **복잡한 파이프라인**을 단계별로 구현할 때 유용
- **동작 방식**
  - 첫 번째 LLMChain 실행 → 출력 생성
  - 출력값을 두 번째 Chain 입력으로 전달
  - 모든 체인을 순차적으로 실행 후 최종 결과 반환
- **활용 예시**
  - "책 제목 생성 → 해당 제목에 맞는 책 소개글 생성 → 책 소개를 요약"
  - "사용자 질의 → SQL 쿼리 생성 → 결과 해석"

RouterChain

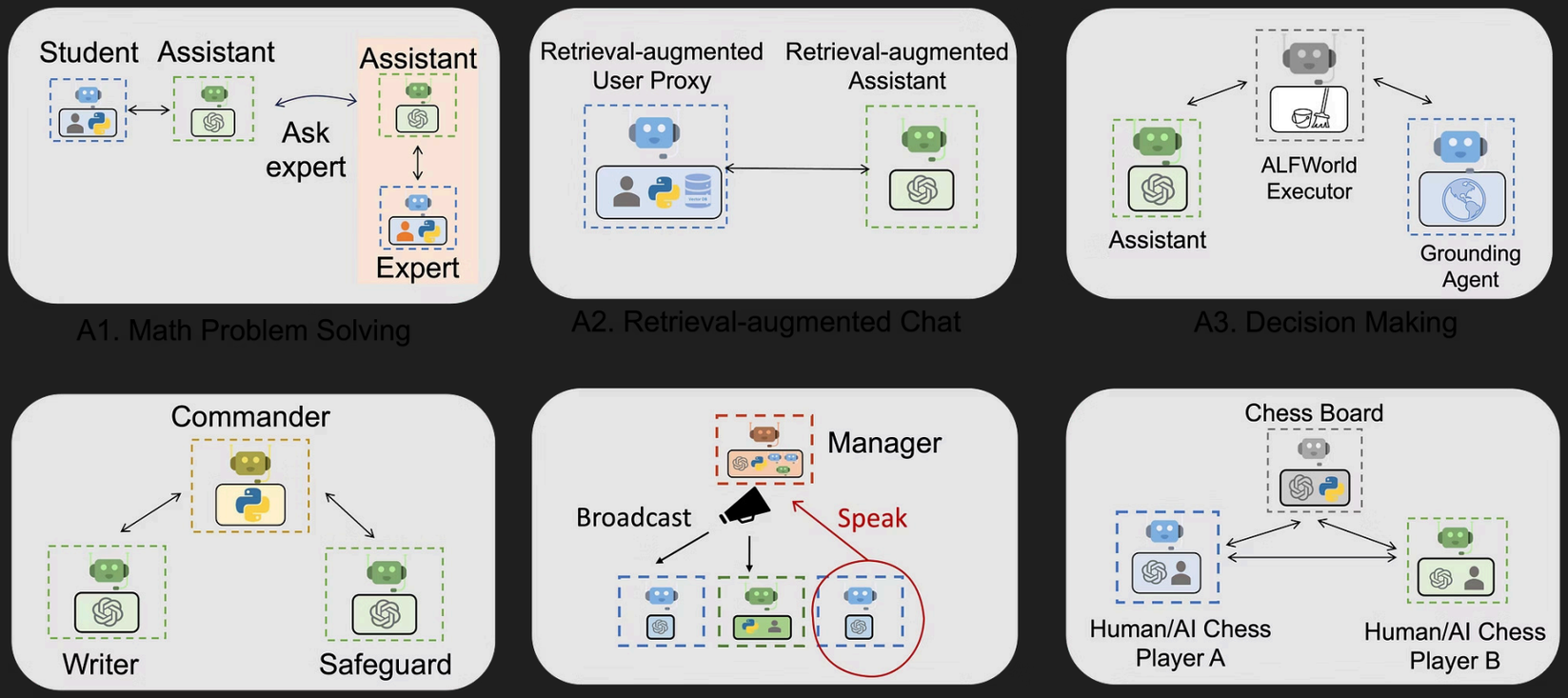
- **개념**
  - 조건에 따라 다른 체인으로 라우팅하는 방식
  - **의사결정 로직**을 포함하여 입력에 맞는 최적의 체인 선택
  - 복잡한 애플리케이션에서 효율적인 제어 가능
- **동작 방식**
  - 입력을 분석하여 어떤 체인을 사용할지 결정 (Router)
  - 선택된 체인 실행
  - 결과 반환
- **활용 예시**
  - **FAQ Bot**:
    - 입력이 "일반 지식 질문" → LLMChain으로 처리
    - 입력이 "데이터베이스 질의" → SQLDatabaseChain으로 라우팅
    - 입력이 "수학 계산" → Python REPL Tool로 라우팅
  - 다국어 입력 분기 (한국어 → 한국어 체인, 영어 → 영어 체인)

기사 요약 체인 만들기 실습

```
from langchain.chains import SequentialChain, LLMChain
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate

llm = OpenAI(temperature=0)

# 첫 번째 체인: 기사 요약
summarize_template = """
다음 뉴스 기사를 3-4문장으로 요약해주세요:
{article}
요약:
"""
summarize_prompt = PromptTemplate(template=summarize_template, input_variables=["article"])
summarize_chain = LLMChain(llm=llm, prompt=summarize_prompt, output_key="summary")

# 두 번째 체인: 핵심 주제 추출
topics_template = """
다음 요약에서 핵심 주제 3가지를 추출해주세요:
{summary}
주제:
"""
topics_prompt = PromptTemplate(template=topics_template, input_variables=["summary"])
topics_chain = LLMChain(llm=llm, prompt=topics_prompt, output_key="topics")

# 체인 연결
sequential_chain = SequentialChain(
 chains=[summarize_chain, topics_chain],
 input_variables=["article"],
 output_variables=["summary", "topics"]
)

# 실행
result = sequential_chain({"article": "여기에 긴 뉴스 기사 내용..."})
print("요약:", result["summary"])
print("주제:", result["topics"])
```

메모리 시스템

대화형 AI에서 메모리의 중요성



A1. Math Problem Solving
A2. Retrieval-augmented Chat
A3. Decision Making
A4. Multi-agent Coding
A5. Dynamic Group Chat
A6. Conversational Chess

대화형 AI에서 맥락(Context) 유지는 자연스러운 상호작용에 필수적

메모리 없는 시스템의 한계:

- 이전 대화 내용 기억 못 함
- 이전 언급 참조 불가
- 정보 반복 필요
- 대화 흐름 단절

메모리 시스템의 이점:

- 자연스러운 대화 흐름 유지
- 사용자 선호도 및 과거 상호작용 기억
- 맥락에 맞는 정확한 응답 생성

LangChain의 메모리 종류

ConversationBufferMemory

모든 대화 내용을 있는 그대로 저장한다.

- 장점: 완전한 대화 기록을 보존한다.
- 단점: 대화가 길어지면 컨텍스트 크기 제한에 도달한다.

ConversationSummaryMemory

긴 대화를 요약하여 저장함으로써 컨텍스트 길이 제한을 해결한다.

- 장점: 긴 대화에서도 핵심 내용을 유지한다.
- 단점: 요약 과정에서 세부 정보가 손실될 수 있다.

ConversationKGMemory

대화에서 추출한 정보를 지식 그래프 형태로 저장한다.

- 장점: 엔티티 간 관계를 유지하며 구조화된 기억을 제공한다.
- 단점: 구현 복잡성이 증가한다.

VectorStoreMemory

대화 내용을 벡터로 변환하여 저장하고 유사성 기반으로 검색한다.

- 장점: 의미적 유사성을 기반으로 참조할 수 있다.
- 단점: 추가적인 벡터 데이터베이스가 필요하다.

실습: 대화형 챗봇 구현하기

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
from langchain.llms import OpenAI

# 메모리 초기화
memory = ConversationBufferMemory()

# 대화 체인 생성
conversation = ConversationChain(
 llm=OpenAI(temperature=0.7),
 memory=memory,
 verbose=True
)

# 첫 번째 대화
response = conversation.predict(input="안녕하세요! 제 이름은 김개발입니다.")
print(response)

# 두 번째 대화 (이전 맥락 참조)
response = conversation.predict(input="제 이름이 뭐였죠?")
print(response)

# 세 번째 대화
response = conversation.predict(input="제가 어떤 분야에 관심이 많을 것 같나요?")
print(response)

# 저장된 메모리 확인
print(memory.buffer)
```

메모리 시스템 고급 기법

윈도우 메모리

가장 최근 N개의 대화만 유지하는 슬라이딩 윈도우 방식이다.

```
from langchain.memory import ConversationBufferWindowMemory

window_memory = ConversationBufferWindowMemory(
    k=5  # 최근 5개 대화만 유지
)
```

요약 메모리

대화가 길어지면 자동으로 요약하여 저장한다.

```
from langchain.memory import ConversationSummaryMemory

summary_memory = ConversationSummaryMemory(
    llm=OpenAI()
)
```

토큰 제한 메모리

최대 토큰 수를 제한하여 컨텍스트 크기를 관리한다.

```
from langchain.memory import ConversationTokenBufferMemory

token_memory = ConversationTokenBufferMemory(
    llm=OpenAI(),
    max_token_limit=1000
)
```

벡터 저장소 메모리

의미 기반 검색을 위해 벡터를 저장한다.

```
from langchain.memory import VectorStoreRetrieverMemory
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

embedder = OpenAIEmbeddings()
vectorstore = Chroma(embedding_function=embedder)
retriever = vectorstore.as_retriever()

vector_memory = VectorStoreRetrieverMemory(
 retriever=retriever
)
```

문서 처리 시스템

Document Loaders 소개

Document Loader는 다양한 형식의 외부 데이터를 LangChain 생태계로 가져오는 인터페이스이다. AI가 다양한 소스의 데이터를 처리해야 하므로, 실제 업무 환경에서 문서 로더의 역할이 중요하다.

PDF 로더

PyPDF, PDFPlumber 등으로 PDF 파일의 텍스트를 추출한다.

Word 로더

docx, doc 파일에서 텍스트 및 메타데이터를 추출한다.

CSV/Excel 로더

표 형식 데이터를 구조화된 형태로 로드한다.

웹 로더

URL에서 웹페이지 콘텐츠를 추출하고 정제한다.

다양한 Document Loader 예시

1. PDF 문서 로딩

```
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("path/to/document.pdf")
pages = loader.load()

# 각 페이지는 Document 객체
print(f"페이지 수: {len(pages)}")
print(f"첫 페이지 내용: {pages[0].page_content[:100]}...")
```

2. 웹페이지 로딩

```
from langchain.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://example.com")
docs = loader.load()
```

3. 디렉토리 로딩

```
from langchain.document_loaders import DirectoryLoader

# 특정 디렉토리의 모든 .txt 파일 로드
loader = DirectoryLoader(
    "./data/",
    glob="**/*.txt",
    show_progress=True
)
docs = loader.load()
```

4. Notion 데이터 로딩

```
from langchain.document_loaders import
NotionDirectoryLoader

loader = NotionDirectoryLoader("./notion_export")
docs = loader.load()
```

Text Splitters의 필요성

Text Splitter는 긴 문서를 LLM 처리 가능한 크기의 청크(chunk)로 분할하는 도구다.

Text Splitter 필요성:

- **토큰 제한:** LLM 컨텍스트 윈도우 크기 제한 존재 (예: GPT-4 8K~32K 토큰)
- **관련성 향상:** 작은 청크 분할로 검색 시 높은 관련성 확보
- **비용 효율성:** 필요 부분만 처리하여 API 호출 비용 절감

이상적 청크 크기:

- 일반적 400~1000 토큰 적절
- 과소 시 맥락 손실, 과대 시 관련성 희석
- 사용 사례별 실험적 최적화 필요

주요 Text Splitter 종류

CharacterTextSplitter

특정 문자를 기준으로 텍스트를 분할하는 기본 분할기.

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    separator="\n\n",
    chunk_size=1000,
    chunk_overlap=200
)
```

RecursiveCharacterTextSplitter

다양한 구분자를 재귀적으로 적용, 자연스러운 텍스트 분할.

```
from langchain.text_splitter import
RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", ".", " "],
    chunk_size=1000,
    chunk_overlap=200
)
```

MarkdownTextSplitter

마크다운 구조를 인식하여 분할.

```
from langchain.text_splitter import MarkdownTextSplitter

splitter = MarkdownTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
```

TokenTextSplitter

특정 토크나이저를 활용, 정확한 토큰 수 기준으로 분할.

```
from langchain.text_splitter import TokenTextSplitter

splitter = TokenTextSplitter(
 encoding_name="cl100k_base", # tiktoken 인코딩
 chunk_size=500,
 chunk_overlap=50
)
```

실습: PDF 문서 로딩 및 분할

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# 1. PDF 로딩
loader = PyPDFLoader("research_paper.pdf")
pages = loader.load()
print(f"총 {len(pages)}페이지 로드됨")

# 2. 텍스트 분할
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n\n", "\n", ".", " "]
)

# 문서를 청크로 분할
chunks = text_splitter.split_documents(pages)
print(f"총 {len(chunks)}개 청크로 분할됨")

# 청크 확인
for i, chunk in enumerate(chunks[:3]):  # 처음 3개 청크만 확인
    print(f"\n--- 청크 {i+1} ---")
    print(f"텍스트: {chunk.page_content[:100]}...")
    print(f"메타데이터: {chunk.metadata}")
```

분할된 청크는 벡터 데이터베이스 저장 또는 LLM 직접 처리에 활용된다.

벡터 데이터베이스

벡터 임베딩의 이해

벡터 임베딩은 텍스트, 이미지 등 데이터를 고차원 벡터 공간에 매핑하여 의미적 유사성을 수치화하는 과정이다.

임베딩의 핵심 개념:

- **차원 축소:** 복잡한 데이터를 고정된 크기 벡터로 변환
- **의미 보존:** 유사한 의미의 텍스트는 벡터 공간에서 가깝게 위치
- **거리 계산:** 코사인 유사도 등으로 벡터 간 유사성 측정

임베딩 모델 종류:

- **OpenAI Embeddings:** text-embedding-ada-002 등
- **HuggingFace Embeddings:** BERT, sentence-transformers 등
- **한국어 특화 모델:** KoBERT, KoSBERT 등

LangChain 임베딩 활용

```
from langchain.embeddings import OpenAIEmbeddings, HuggingFaceEmbeddings

# OpenAI 임베딩 모델 사용
openai_embeddings = OpenAIEmbeddings(
 model="text-embedding-ada-002", # 또는 최신 모델
 openai_api_key="sk-your-api-key"
)

# 텍스트 임베딩 생성
text = "LangChain은 LLM 애플리케이션 개발을 위한 프레임워크입니다."
embedding_vector = openai_embeddings.embed_query(text)
print(f"임베딩 벡터 차원: {len(embedding_vector)}")

# HuggingFace 임베딩 모델 사용 (한국어 특화)
korean_embeddings = HuggingFaceEmbeddings(
 model_name="jhgan/ko-sroberta-multitask", # 한국어 임베딩 모델
 model_kwargs={'device': 'cpu'}
)

# 한국어 텍스트 임베딩
korean_text = "인공지능 기술의 발전은 사회에 큰 영향을 미치고 있습니다."
korean_embedding = korean_embeddings.embed_query(korean_text)
print(f"한국어 임베딩 벡터 차원: {len(korean_embedding)}")
```

벡터 데이터베이스 개요

벡터 데이터베이스는 임베딩 벡터의 효율적 저장 및 검색을 위한 특수 데이터베이스.

주요 기능

- 유사도 기반 검색 (Similarity Search)
- K-최근접 이웃(KNN) 검색
- 메타데이터 필터링
- 벡터 연산 최적화

로컬 벡터 DB

- **FAISS:** Facebook AI의 고성능 벡터 검색
- **Chroma:** 간단한 인메모리 벡터 저장소
- **Qdrant:** 로컬 또는 서버 모드 지원

클라우드 벡터 DB

- **Pinecone:** 완전 관리형 벡터 검색
- **Weaviate:** 지식 그래프 + 벡터 검색
- **Milvus:** 대규모 벡터 검색 지원

실습: Chroma 벡터 DB 구축하기

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

# 1. 문서 로딩
loader = TextLoader("data.txt")
documents = loader.load()

# 2. 문서 분할
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
docs = text_splitter.split_documents(documents)

# 3. 임베딩 모델 초기화
embeddings = OpenAIEmbeddings()

# 4. Chroma 벡터 DB 생성
vectordb = Chroma.from_documents(
 documents=docs,
 embedding=embeddings,
 persist_directory="./chroma_db" # 디스크에 저장
)

# 5. 유사성 검색 수행
query = "인공지능의 윤리적 문제는 무엇인가요?"
docs = vectordb.similarity_search(query, k=3) # 상위 3개 문서 검색

# 6. 검색 결과 확인
for i, doc in enumerate(docs):
 print(f"\n--- 검색 결과 {i+1} ---")
 print(f"내용: {doc.page_content[:150]}...")
 print(f"메타데이터: {doc.metadata}")

# 7. 벡터 DB 저장
vectordb.persist()
```

벡터 데이터베이스 고급 기능

메타데이터 필터링

벡터 유사성 기반 검색에 메타데이터 조건을 추가하여 결과 범위 제한.

```
# 특정 파일 또는 페이지만 검색
filtered_docs = vectordb.similarity_search(
    query="인공지능 윤리",
    k=5,
    filter={"source": "ethics_paper.pdf"}
)

# 날짜 범위로 필터링
filtered_docs = vectordb.similarity_search(
    query="최신 AI 트렌드",
    filter={"date": {"$gte": "2023-01-01"}}
)
```

MMR 검색 (Maximum Marginal Relevance)

검색 결과의 다양성과 관련성을 동시에 고려하여 균형 조정.

```
# 다양성과 관련성 균형 조정
mmr_docs = vectordb.max_marginal_relevance_search(
    query="딥러닝 알고리즘",
    k=5,
    fetch_k=20,  # 후보 풀 크기
    lambda_mult=0.5  # 다양성 가중치 (0-1)
)
```

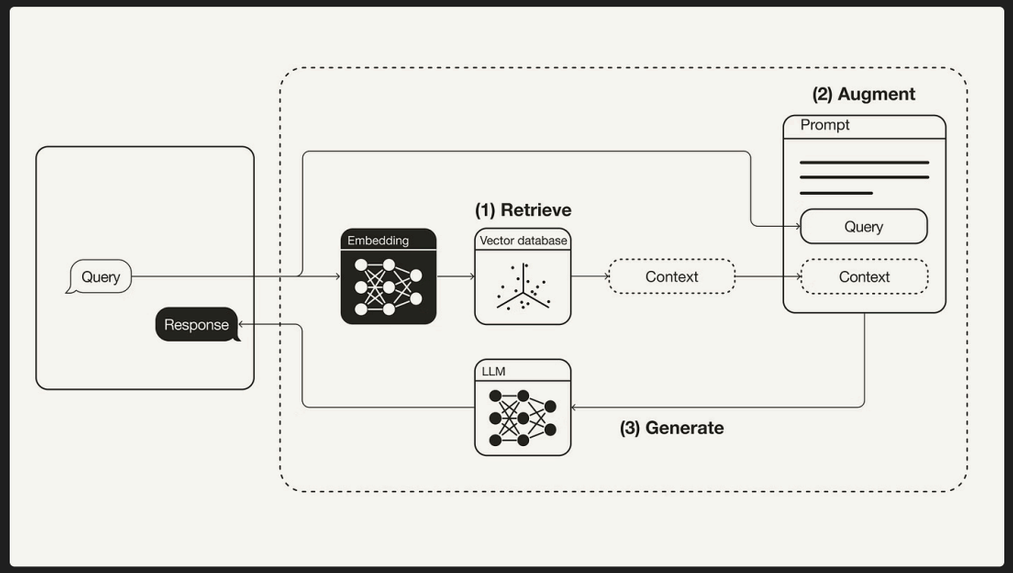
하이브리드 검색

키워드 검색과 벡터 검색을 결합하여 활용.

```
# 일부 벡터 DB에서 지원
hybrid_results = weaviate_db.hybrid_search(
 query="트랜스포머 모델 구조",
 k=5,
 alpha=0.5 # BM25와 벡터 검색 비율
)
```

LangChain + 벡터DB 통합

RetrievalQA의 개념



RetrievalQA는 검색 기반 질의응답 시스템이다. 대규모 문서에서 질문 관련 정보를 검색하고 이를 바탕으로 답변을 생성한다.

작동 원리:

1. **질문 분석:** 사용자 질문을 이해한다.
2. **문서 검색:** 관련 문서 조각을 검색한다.
3. **컨텍스트 구성:** 검색 결과를 LLM 입력으로 구성한다.
4. **답변 생성:** 검색된 정보를 바탕으로 응답을 생성한다.

장점:

- 방대한 문서 처리가 가능하다.
- 최신 정보 반영이 가능하다 (학습 데이터 이후 정보).
- 소스 인용 및 근거 제시가 가능하다.
- 환각(Hallucination)을 감소시킨다.

단순 LLM vs 검색 기반 QA 비교

단순 LLM 질의응답

장점:

- 구현 간단, 외부 데이터 불필요.
- 일반적 질문에 빠르게 응답.

단점:

- 학습 데이터 이후 정보 부족.
- 특정 도메인 지식에 제한적.
- 환각(Hallucination) 발생 가능성 높음.
- 소스 인용 어려움.

검색 기반 질의응답 (RAG)

장점:

- 최신 정보 및 전문 지식 제공.
- 정보 소스 추적 및 인용 가능.
- 환각 현상 감소.
- 비공개 문서 활용 가능.

단점:

- 구현 복잡성 증가.
- 인프라 요구사항 증가.
- 검색 품질에 의존적.

검색 증강 생성(RAG, Retrieval-Augmented Generation)은 LLM의 한계를 극복하고, 정확하고 신뢰할 수 있는 응답을 제공하는 핵심 패턴이다.

RetrievalQA 구현하기

```
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# 1. 문서 로딩 및 분할
loader = PyPDFLoader("company_annual_report.pdf")
documents = loader.load()

text_splitter = RecursiveCharacterTextSplitter(
 chunk_size=1000,
 chunk_overlap=200
)
chunks = text_splitter.split_documents(documents)

# 2. 벡터 데이터베이스 구축
embeddings = OpenAIEmbeddings()
vectordb = Chroma.from_documents(chunks, embeddings)

# 3. RetrievalQA 체인 생성
retriever = vectordb.as_retriever(
 search_type="similarity",
 search_kwargs={"k": 4} # 상위 4개 문서 검색
)

qa_chain = RetrievalQA.from_chain_type(
 llm=OpenAI(temperature=0),
 chain_type="stuff", # 또는 "map_reduce", "refine"
 retriever=retriever,
 return_source_documents=True # 소스 문서 반환
)

# 4. 질의응답 실행
query = "이 회사의 2023년 주요 성과는 무엇인가요?"
result = qa_chain({"query": query})

print(f"질문: {query}")
print(f"답변: {result['result']}")
print("\n참조 문서:")
for i, doc in enumerate(result["source_documents"]):
 print(f"\n문서 {i+1}:")
 print(f"내용: {doc.page_content[:100]}...")
 print(f"출처: {doc.metadata}")
```

RetrievalQA 고급 체인 타입

1

stuff

모든 관련 문서를 하나로 결합하여 LLM에 전달하는 가장 단순한 방식입니다. 구현이 간단하고 LLM 호출 횟수가 적다는 장점이 있으나, 컨텍스트 길이 제한으로 대량 문서에는 부적합합니다.

2

map_reduce

각 문서에 대해 개별적으로 답변을 생성한 후 최종 응답으로 종합합니다. 대량 문서 처리가 가능하고 병렬화에 유리하지만, 여러 번의 LLM 호출이 필요하며 정보 손실 가능성이 있습니다.

3

refine

각 문서를 순차적으로 처리하며 답변을 점진적으로 개선합니다. 문서 간 연결성을 유지하고 포괄적인 답변을 제공하지만, 문서 수에 비례하여 LLM 호출이 늘어나고 순차 처리로 인해 속도가 느립니다.

Tool & Agent

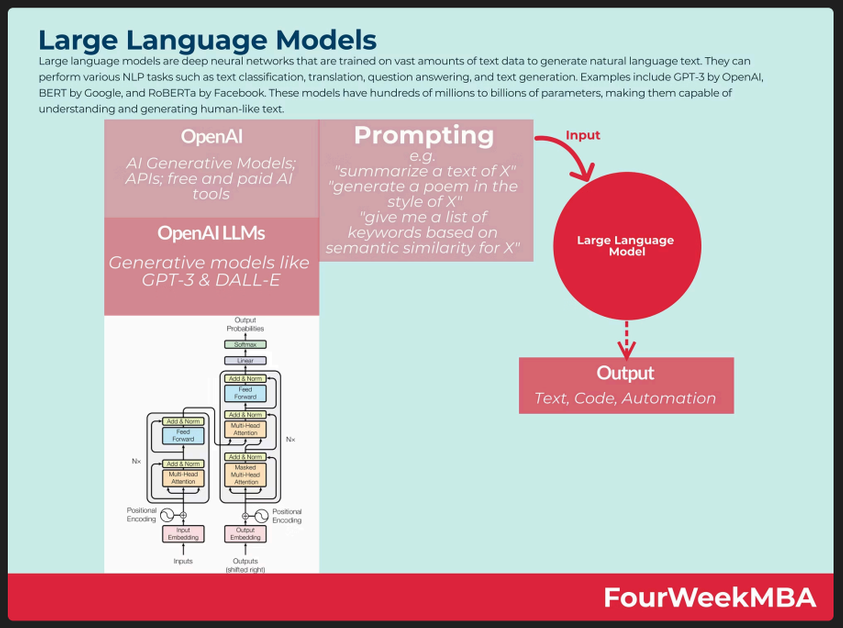
Tool의 개념과 종류

Tool은 LLM이 외부 시스템이나 데이터에 접근하는 인터페이스이다. LLM 능력을 확장하여 계산, 검색, 데이터베이스 쿼리 등 작업을 수행할 수 있도록 한다.

주요 Tool 종류:

- **검색 도구:** 웹 검색, 문서 검색
- **계산 도구:** 계산기, 수학 연산
- **코드 실행:** Python REPL, 코드 인터프리터
- **데이터베이스:** SQL 쿼리, 데이터베이스 접근
- **API 호출:** 외부 서비스 연동 (날씨, 주식, 뉴스 등)
- **파일 시스템:** 파일 읽기/쓰기

Tool은 사용 방법과 입/출력 형식에 대한 명확한 설명을 포함해야 한다.



Agent의 이해

Agent는 주어진 목표 달성을 위해 어떤 도구를 언제 사용할지 결정하는 지능형 시스템. 사용자 의도를 이해하고 적절한 도구 선택으로 복잡한 작업을 수행

입력 처리

사용자 요청 분석 및 이해.

사고 (Thought)

작업 분해 및 실행 계획 수립.

도구 선택

적절한 도구 선택 및 입력 구성.

행동 (Action)

선택한 도구 실행.

관찰

도구 실행 결과 분석.

응답 생성

최종 결과 종합 및 응답.

이 과정은 **ReAct (Reasoning + Acting)** 패턴으로, 에이전트가 사고와 행동을 반복하며 목표에 도달

Tool 및 Agent 구현

```
from langchain.agents import initialize_agent, Tool
from langchain.agents import AgentType
from langchain.llms import OpenAI
from langchain.tools import BaseTool
from langchain.chains import LLMMathChain
from typing import Type

# 1. 계산 도구 생성
llm = OpenAI(temperature=0)
llm_math_chain = LLMMathChain.from_llm(llm=llm, verbose=True)

# 2. 검색 도구 생성 (DuckDuckGo 검색)
from langchain.utilities import DuckDuckGoSearchAPIWrapper
search = DuckDuckGoSearchAPIWrapper()

def ddg_search(query):
 return search.run(query)

# 3. 커스텀 도구 생성 (날씨 정보)
class WeatherTool(BaseTool):
 name = "weather_tool"
 description = "현재 날씨 정보를 조회합니다. 입력은 '도시명'이어야 합니다."

 def _run(self, city: str) -> str:
 # 실제로는 날씨 API를 호출하지만, 예시로 하드코딩
 return f"{city}의 현재 날씨는 맑고 기온은 23도입니다."

 async def _arun(self, city: str) -> str:
 return self._run(city)
```

```
# 4. 도구 목록 정의
tools = [
 Tool(
 name="Calculator",
 func=llm_math_chain.run,
 description="복잡한 수학 계산이 필요할 때 사용합니다."
 ),
 Tool(
 name="Search",
 func=ddg_search,
 description="최신 정보나 일반적인 질문에 대한 답을 찾을 때 에이전트가 사용합니다."
 ),
 WeatherTool()
]

# 5. 에이전트 초기화
agent = initialize_agent(
 tools,
 llm,
 agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, # ReAct 패턴 사용
 verbose=True
)

# 6. 에이전트 실행
response = agent.run("서울의 현재 날씨는 어떻고, 섭씨 23도는 화씨로 몇 도인가요?")
print(response)
```

복합 시스템 설계

복합 시스템 설계 개요

LLM, 데이터베이스, API를 조합하여 단순 텍스트 생성을 넘어 실제 데이터 기반의 AI 애플리케이션 구축

- **LLM:** 자연어 이해 및 결과 생성
- **데이터베이스 (DB):** 정형 데이터 조회 및 관리
- **API:** 실시간 외부 데이터 및 서비스 연동

LangChain을 활용한 데이터베이스 연동

LangChain의 SQLDatabaseChain은 자연어 질의를 SQL 쿼리로 자동 변환

주요 특징:

- **SQLAlchemy 통합:** SQLAlchemy와 연동하여 다양한 관계형 DB를 지원
- **자동 스키마 파악 및 질의:** 데이터베이스 스키마를 자동으로 이해하고, 사용자 질문에 맞는 SQL 쿼리 생성
- **실무 예시 적용:** 비즈니스 질문에 즉시 응답하여 데이터 기반 의사결정을 도움

코드 스니펫: SQLDatabaseChain 구현

```
from langchain.chains import SQLDatabaseChain
from langchain.sql_database import SQLDatabase
from langchain.llms import OpenAI

# LLM 초기화 (예시: OpenAI)
llm = OpenAI(temperature=0)

# 1. SQLite 데이터베이스 연결
# 실제 사용 시에는 PostgreSQL, MySQL 등으로 변경 가능
db = SQLDatabase.from_uri("sqlite:///sales.db")

# 2. SQLDatabaseChain 생성
# verbose=True로 설정하면 LLM이 생성한 SQL 쿼리 확인 가능
db_chain = SQLDatabaseChain(llm=llm, database=db, verbose=True)

# 3. 자연어 질의 실행
query = "지난주 매출 합계를 알려줘"
response = db_chain.run(query)

print(f"질문: {query}")
print(f"답변: {response}")
```

외부 API 연동

실시간 데이터(날씨, 환율, 주식 등)를 LLM과 결합하여 애플리케이션 기능을 확장

1 다양한 API 연동

광범위한 실시간 데이터로 LLM의 정보 범위 확장

2 LangChain 기능 활용

LangChain의 API 연동 기능을 사용하여 통합

예를 들어 "오늘 서울 날씨에 맞는 옷차림 추천"과 같은 복잡한 질문에 정확히 응답

```
from langchain.utilities import RequestsWrapper
requests = RequestsWrapper()
weather =
requests.get("https://api.weather.com/seoul/today")
```

DB + API 통합 워크플로우

데이터베이스 정보와 외부 API 데이터를 조합하여 LLM이 더욱 풍부하고 상황에 맞는 답변 제공

예시 시나리오:

- **사용자:** "이번 주 날씨와 매출 데이터를 비교해줘"
- **LLM 동작:**
  a. 데이터베이스에서 이번 주 매출 데이터 조회
  b. 날씨 API 호출로 날씨 데이터 확보
  c. 두 데이터 통합하여 분석 및 결과 반환
- **결과 예시:** "이번 주 비 오는 날 매출이 20% 증가했습니다."

LLM의 한계를 넘어서 실시간 정보와 기업 내부 데이터를 결합한 고도화된 비즈니스 통찰력을 제공

실습: DB + API 통합

실습 단계

01

SQLite 매출 데이터베이스 생성

가상의 매출 데이터를 포함하는 SQLite 데이터베이스를 설정합니다.

02

날씨 API 호출 함수 구현

외부 날씨 API를 호출하여 특정 도시의 날씨 정보를 가져오는 함수를 작성합니다.

03

LangChain으로 DB + API 결과 통합 체인 제작

LangChain의 SQLDatabaseChain과 API 호출 도구를 결합하여 사용자 질문에 따라 DB와 API 데이터를 함께 활용하는 체인을 만듭니다.

04

사용자 질문 테스트

"비 오는 날 매출 추이 알려줘"와 같은 질문으로 통합 시스템을 테스트하고 결과를 확인합니다.

실습 결과 예시

입력: "이번 주 날씨와 매출 데이터 비교해줘"

```
출력:
월요일(맑음): 매출 120만원
화요일(비): 매출 180만원
수요일(흐림): 매출 150만원
목요일(맑음): 매출 130만원
금요일(비): 매출 200만원

→ 분석 결과, 비 오는 날 매출이 맑은 날보다 약 30% 더 높게 나타났습니다.
```